



CONTENTS

1	Exponents	3
1.1	Identities for Exponents	3
2	Compound Interest	7
2.1	An example with annual interest payments	7
2.2	Exponential Growth	8
2.3	Sensitivity to interest rate	9
3	Logarithms	11
3.1	Logarithms in Python	11
3.2	Logarithm Identities	12
3.3	Changing Bases	13
3.4	Natural Logarithm	13
3.5	Logarithms in Spreadsheets	13
4	Exponential Decay	15
4.1	Radioactive Decay	16
4.2	Model Exponential Decay	17
5	Inverse Trigonometric Functions	19
5.1	Derivatives of Inverse Trigonometric Functions	20
5.2	Practice	20
6	Introduction to Graphs	21
6.1	Introduction	21

6.2	Finding Good Paths	23
6.3	Graphs in Python	24
7	Laws, Paradoxes, Thought Experiments, and the Illusions	29
7.1	Mathematical Paradoxes	30
7.1.1	Russell's Paradox	30
7.1.2	Simpson's Paradox	31
7.1.3	Monty Hall Problem	33
7.1.4	Birthday Problem	33
7.1.5	Zeno's Paradoxes	35
7.2	Physics and Time Travel Paradoxes	36
7.2.1	Schrödinger's Cat	36
7.2.2	Time Travel Paradoxes	37
7.3	Architecture and Geometry Paradoxes	38
7.3.1	Penrose Steps	38
7.3.2	Mobius Strip	39
7.3.3	Coastline paradox	39
7.4	Computer Science Paradoxes and Laws	40
7.4.1	Moravec's Paradox	40
7.4.2	Neural Scaling Law	41
A	Answers to Exercises	43
	Index	45

Exponents

Let's take quick look at exponents. Ancient scientists started coming up with a lot of formulas that involved multiplying the same number several times. For example, if they knew that a sphere was r centimeters in radius, its volume in milliliters was

$$V = \frac{4}{3} \times \pi \times r \times r \times r$$

They did two things to make the notation less messy. First, they decided that if two numbers were written next to each other, the reader would assume that meant "multiply them". Second, they came up with the exponent, a little number that was lifted off the baseline of the text, that meant "multiply it by itself". For example 5^3 was the same as $5 \times 5 \times 5$.

Now, the formula for the volume of a sphere is written

$$V = \frac{4}{3} \pi r^3$$

Tidy, right? In an exponent expression like this, we say that r is *the base* and 3 is *the exponent*.

1.1 Identities for Exponents

What about exponents of exponents? What is $(5^3)^2$?

$$(5^3)^2 = (5 \times 5 \times 5)^2 = (5 \times 5 \times 5)(5 \times 5 \times 5) = 5^6$$

In general, for any a , b , and c :

$$(a^b)^c = a^{(bc)}$$

If you have $(5^3)(5^4)$, that is just $5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5$ or 5^7

The general rule is that for any a , b , and c

$$(a^b)(a^c) = a^{(b+c)}$$

Mathematicians *love* this rule, so we keep extending the idea of exponents to keep this rule true. For example, at some point, someone asked “What about 5^0 ?” According to the rule, 5^2 must equal $5^{(2+0)}$ which must equal $(5^2)(5^0)$. Thus, 5^2 must be 1. So mathematicians declared “Anything to the power of 0 is 1”.

We don’t typically assume that $0^0 = 1$. It is just too weird. So, we say that for any a not equal to zero,

$$a^0 = 1$$

What about $5^{(-2)}$? By our beloved rule, we know that $(5^{-2})(5^5)$ must be equal to 5^3 , right? So 5^{-2} must be equal to $\frac{1}{5^2}$.

We say that for any a not equal to zero and any b ,

$$a^{-b} = \frac{1}{a^b}$$

This makes dividing one exponential expression by another (with the same base) easy:

$$\frac{a^b}{a^c} = a^{(b-c)}$$

We often say “cancel out” for this. Here, we can “cancel out” x^2 :

$$\frac{x^5}{x^2} = x^3$$

What about $5^{\frac{1}{3}}$? By the beloved rule, we know that $5^{\frac{1}{3}}5^{\frac{1}{3}}5^{\frac{1}{3}}$ must equal 5^1 . Thus, $5^{\frac{1}{3}} = \sqrt[3]{5}$.

We say that for any a and b not equal to zero and any c greater than zero,

$$a^{\frac{b}{c}} = \left(\sqrt[c]{a}\right)^b = \left(\sqrt[c]{a^b}\right)$$

Before you go on to the exercises, note that the beloved rule demands a common base.

- We can combine these: $(5^2)(5^4) = 5^6$
- We cannot combine: $(5^2)(3^5)$

With that said, we note for any a, b , and c :

$$(ab)^c = (a^c)(b^c)$$

So, for example, if we were asked to simplify $(3^4)(6^2)$, we would note that $6 = 2 \times 3$, so

$$(3^4)(6^2) = (3^4)(3^2)(2^2) = (3^6)(2^2)$$

If these ideas are new to you (or maybe you are just having trouble remembering them), watch the Khan Academy's **Intro to rational exponents** video at <https://youtu.be/1ZfXc4nHooo>.

CHAPTER 2

Compound Interest

When you loan money to someone, you typically charge them some sort of interest. The most common loan of this sort is what the bank calls a “savings account”. Any money you put in the account is loaned to the bank. The bank then lends it to someone else, who pays interest to the bank. The bank gives some of that interest to you. However, what if you leave the interest in your account? And you start making *interest on the interest*? This is known as *compound interest*.

2.1 An example with annual interest payments

Let’s say that you put \$1000 in a savings account that pays 6% interest every year. How much money would you have after 12 years? Let’s make a spreadsheet.

	A	B	C
1	Interest Rate	6.00%	
2			
3	After year:	Interest	Balance
4	0	\$0.00	1000
5	1	\$60.00	\$1,060.00

Create a new spreadsheet and edit the cells to look like this. All the cells in rows 1 - 4 are just values: just type in what you see.

The fifth row is all formulas:

After year	Interest	Balance
= A4 + 1	= B\$1 * C4	= C4 + B5

The interest rate field should be formatted as a percentage. One thing to know when dealing with percentages in the spreadsheet: If the field says “600%”, its value is 6.

The cells in the Interest and Balance column should be formatted as currency.

You are about to make several copies of the cells in the fifth row, so make sure they look right.

Click on A5 and shift-click on C5 to select all three cells. Drag the lower-right corner down to fill the rows 6 - 15.

A5:C16 fx = A4 + 1			
	A	B	C
1	Interest Rate	6.00%	
2			
3	After year	Interest	Balance
4	0	\$0.00	1000
5	1	\$60.00	\$1,060.00
6	2	\$63.60	\$1,123.60
7	3	\$67.42	\$1,191.02
8	4	\$71.46	\$1,262.48
9	5	\$75.75	\$1,338.23
10	6	\$80.29	\$1,418.52
11	7	\$85.11	\$1,503.63
12	8	\$90.22	\$1,593.85
13	9	\$95.63	\$1,689.48
14	10	\$101.37	\$1,790.85
15	11	\$107.45	\$1,898.30
16	12	\$113.90	\$2,012.20
17			

Look at the numbers. The first interest payment is \$60, but the last is \$113.90. Your balance has more than doubled!

2.2 Exponential Growth

We figured this out numerically by repeatedly multiplying the balance by the interest rate. What if you wanted to know what the balance would be n years after investing P_0 dollars with an annual interest rate of r ? (Note that r in our example would be 0.06, not 6.0.)

Each year, the balance is multiplied by $1+r$, so after one year, P_0 would become $P_0 \times (1+r)$. The next year you would multiply this number by $(1+r)$ again: $P_0 \times (1+r) \times (1+r)$. The next year? $P_0 \times (1+r) \times (1+r) \times (1+r)$ See the pattern? P_n is this balance after n years, then

$$P_n = P_0(1+r)^n$$

Because n is an exponent, we call this *exponential growth*. There are few things as terrifying to a scientist as the phrase “The population is undergoing exponential growth”.

If the principal is not compounded annually, we can modify the equation to account for that by changing n .

$$n = mt$$

Where t is the frequency of the compounding, called the period: (1: annually, 12: monthly, 52: weekly, 365: daily), and m is the number of periods.

2.3 Sensitivity to interest rate

For most people, the first surprising thing about compound interest is how quickly your money grows after a few years. The second thing that is surprising is how much difference a small change in the percentage rate makes.

Let's add another set of columns that shows what happens to your money if you convince the bank to pay you 8% instead of 6%.

Copy everything from columns B and C:

	A	B	C	D	E
1	Interest Rate	6.00%		6.00%	
2					
3	After year	Interest	Balance	Interest	Balance
4	0	\$0.00	1000	\$0.00	1000
5	1	\$60.00	\$1,060.00	\$60.00	\$1,060.00
6	2	\$63.60	\$1,123.60	\$63.60	\$1,123.60
7	3	\$67.42	\$1,191.02	\$67.42	\$1,191.02
8	4	\$71.46	\$1,262.48	\$71.46	\$1,262.48
9	5	\$75.75	\$1,338.23	\$75.75	\$1,338.23
10	6	\$80.29	\$1,418.52	\$80.29	\$1,418.52
11	7	\$85.11	\$1,503.63	\$85.11	\$1,503.63
12	8	\$90.22	\$1,593.85	\$90.22	\$1,593.85
13	9	\$95.63	\$1,689.48	\$95.63	\$1,689.48
14	10	\$101.37	\$1,790.85	\$101.37	\$1,790.85
15	11	\$107.45	\$1,898.30	\$107.45	\$1,898.30
16	12	\$113.90	\$2,012.20	\$113.90	\$2,012.20
17					

Now edit the second interest rate to be 8%:

	A	B	C	D	E
1	Interest Rate	6.00%		8.00%	
2					
3	After year	Interest	Balance	Interest	Balance
4	0	\$0.00	1000	\$0.00	1000
5	1	\$60.00	\$1,060.00	\$80.00	\$1,080.00
6	2	\$63.60	\$1,123.60	\$86.40	\$1,166.40
7	3	\$67.42	\$1,191.02	\$93.31	\$1,259.71
8	4	\$71.46	\$1,262.48	\$100.78	\$1,360.49
9	5	\$75.75	\$1,338.23	\$108.84	\$1,469.33
10	6	\$80.29	\$1,418.52	\$117.55	\$1,586.87
11	7	\$85.11	\$1,503.63	\$126.95	\$1,713.82
12	8	\$90.22	\$1,593.85	\$137.11	\$1,850.93
13	9	\$95.63	\$1,689.48	\$148.07	\$1,999.00
14	10	\$101.37	\$1,790.85	\$159.92	\$2,158.92
15	11	\$107.45	\$1,898.30	\$172.71	\$2,331.64
16	12	\$113.90	\$2,012.20	\$186.53	\$2,518.17
17					

Logarithms

After the world created exponents, it needed the opposite. We could talk about the quantity $? = 2^3$, that is, “What is the product of 2 multiplied by itself three times?” We needed some way to talk about $2^? = 8$, that is, “2 to the what is 8?” This is why we developed the logarithm.

Here is an example:

$$\log_2 8 = 3$$

In English, you would say, “The logarithm base 2 of 8 is 3.”

The base (2, in this case) can be any positive number. The argument (8, in this case) can also be any positive number.

Try this one: What is the logarithm base 2 of 1/16?

You know that $2^{-4} = \frac{1}{16}$, so $\log_2 \frac{1}{16} = -4$.

3.1 Logarithms in Python

Most calculators have pretty limited logarithm capabilities, but python has a nice `log` function that lets you specify both the argument and the base. Start python, import the `math` module, and try taking a few logarithms:

```
1 >>> import math
2 >>> math.log(8,2)
3 3.0
4 >>> math.log(1/16, 2)
5 -4.0
```

Let’s say that a friend offers you 5% interest per year on your investment for as long as you want. You wonder, “How many years before my investment is 100 times as large?” You can solve this problem with logarithms:

```
1 >>> math.log(100, 1.05)
2 94.3872656381287
```

If you leave your investment with your friend for 94.4 years, the investment will be worth 100 times what you put in.

3.2 Logarithm Identities

The logarithm is defined this way:

$$\log_b a = c \iff b^c = a$$

Notice that the logarithm of 1 is always zero, and $\log_b b = 1$.

The logarithm of a product:

$$\log_b ac = \log_b a + \log_b c$$

This follows from the fact that $b^{a+c} = b^a b^c$. What about a quotient?

$$\log_b \frac{a}{c} = \log_b a - \log_b c$$

Exponents?

$$\log_b (a^c) = c \log_b a$$

Notice that because logs and exponents are the opposite of each other, they can cancel each other out:

$$b^{\log_b a} = a$$

and

$$\log_b (b^a) = a$$

3.3 Changing Bases

We mentioned that most calculators have pretty limited logarithm capabilities. Most calculators don't allow you to specify what base you want to work with. All scientific calculators have a button for "log base 10". So, you need to know how to use that button to get logarithms for other bases. Here is the change-of-base identity:

$$\log_b a = \frac{\log_c a}{\log_c b}$$

For example, if you wanted to find $\log_2 8$, you would ask the calculator for $\log_{10} 8$, then divide that by $\log_{10} 2$. You should get 3.

3.4 Natural Logarithm

When you learn about circles, you are told that the circumference of a circle is about 3.141592653589793 times its diameter. Because we use this unwieldy number a lot, we give it a name: We say "The circumference of a circle is π times its diameter."

There is a second unwieldy number that we will eventually use a great deal in solving problems. It is about 2.718281828459045 (but the digits actually go on forever, just like π). We call this number e . (We are not going to talk about why e is special quite yet, but we will soon.)

Most calculators have a button labeled "ln". That is the *natural logarithm* button. It takes the log in base e .

Similarly, in python, if you don't specify a base, the logarithm is done in base e :

```
1 >>> math.log(10)
2 2.302585092994046
3 >>> math.log(math.e)
4 1.0
```

3.5 Logarithms in Spreadsheets

Spreadsheets have three log functions:

- LOG takes both the argument and the base. LOG(8,2) returns 3.
- LOG10 takes just the argument and uses 10 as the base.

- LN takes just the argument and uses e as the base.

Here is a plot from a spreadsheet of a graph of $y = \text{LOG}(x, 2)$.

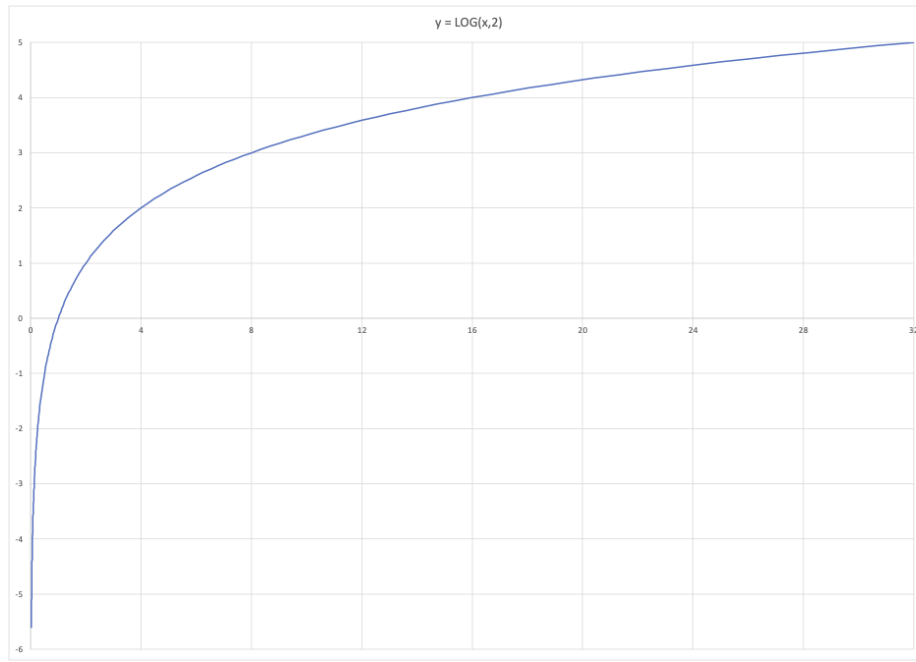


Figure 3.1: A graph of log base 2.

Spreadsheets also have the function $\text{EXP}(x)$, which returns e^x . For example, $\text{EXP}(2)$ returns 7.38905609893065.

CHAPTER 4

Exponential Decay

In a previous chapter, we saw that an investment of P getting compound interest with an annual interest rate of r , grows exponentially. At the end of year t , your balance would be

$$P(1 + r)^t$$

Because r is positive, this number grows as time passes. You get a nice exponential growth curve that looks something like this:

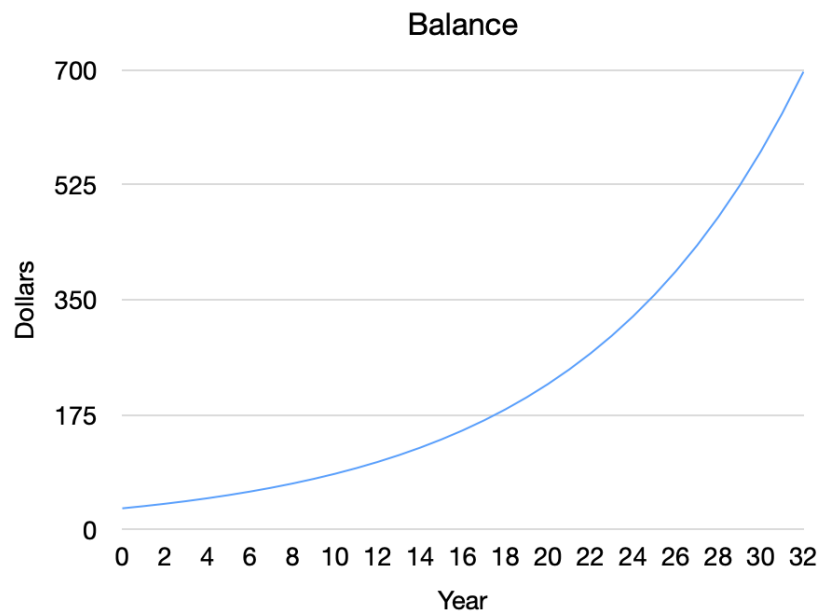


Figure 4.1: A diagram showing (exponential) compound interest.

This is \$30 invested with a 10% annual interest rate. So, the formula for the balance after t years would be

$$(30)(1.1)^t$$

What if r were negative? This would be *exponential decay*.

4.1 Radioactive Decay

Until around 1970, there were companies making watches whose faces and hands were coated with radioactive paint. The paint usually contained radium. When a radium atom decays, it gives off some energy, loses two protons and two neutrons, and becomes a different element (radon). Some of the energy given off is visible light. Thus, these watches glow in the dark.

How many of the radium atoms in the paint decay each century? About 4.24%.

Notice the quantity of atoms lost is proportional to the number of atoms you have. This is exponential decay. If we assume that we start with a million radium atoms, the number of atoms decreases over time like this:

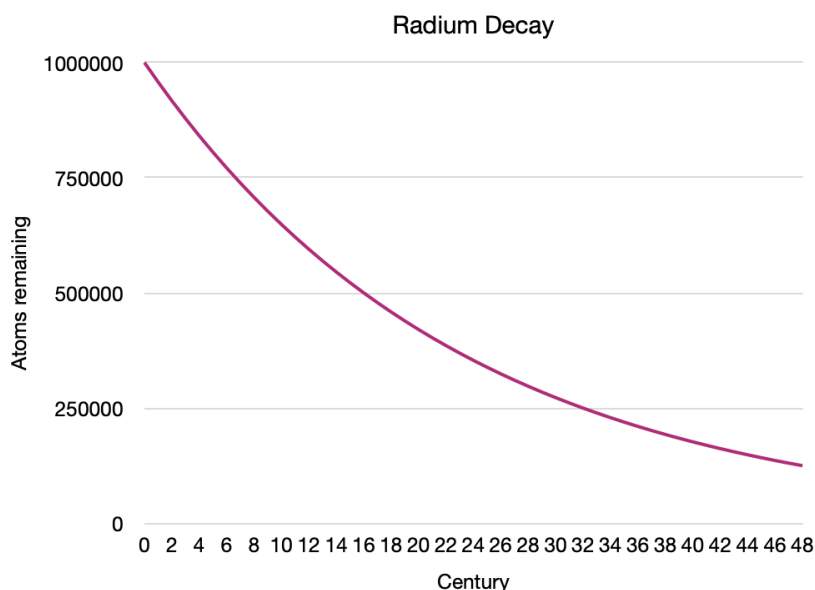


Figure 4.2: A diagram showing the decay and half life of radium.

- We start with 1,000,000 atoms.
- At 16 centuries, we have only 500,000 (half as many) left.
- 16 centuries after that, we have only 250,000 (half again) left.
- 16 centuries after that, we have only 125,000 (half again) left.

A nuclear chemist would say that radium has a *half-life* of 1,600 years; its lifespan decreases by half its original amount every 1,600 years. Note that this means that if you bought a watch with glowing hands in 1960, it will be glowing half as brightly in the year 3560.

How do we calculate the amount of radium left at the end of century t ? If you start with P atoms, at the end of the t -th century, you will have

$$P(1 - 0.0424)^t$$

This is exponential decay.

4.2 Model Exponential Decay

Let's say you get hired to run a company with 480,000 employees. Each year, $1/8$ of your employees leave the company for one reason or another (retirement, quitting, etc.). For some reason, you never hire any new employees.

Make a spreadsheet that indicates how many of the original 480,000 employees will still be around at the end of each year for the next 12. Next, make a bar graph from that data.

Inverse Trigonometric Functions

Recall from the chapter on functions that an inverse of a function is a machine that turns y back into x . The inverses of trigonometric functions are essential to solving certain integrals (you will learn in a future chapter why integrals are useful — for now, trust us that they are!). Let's begin by discussing the sin function and its inverse, $\sin^{-1}$, also called arcsin. (Note: this is not saying sin to the power of -1 , it is a method of stating the inverse of the function).

Examine the graph of $\sin x$ in figure 5.1. See how the dashed horizontal line crosses the function at many points? This means the function $\sin x$ is not one-to-one. In other words, there is not a unique x -value for every y -value. This means that if we do not restrict the domain of $\arcsin x$, the result will not be a function (see figure 5.2). In figure 5.2, you can see that just reflecting the graph across $y = x$ fails the vertical line test: an x value has more than one y value.

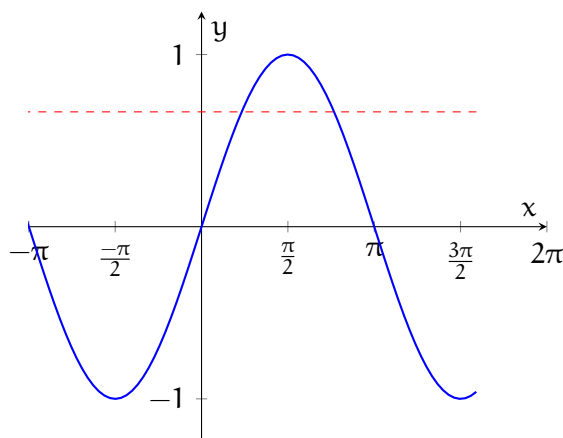


Figure 5.1: The horizontal line $y = \frac{2}{3}$ crosses $y = \sin x$ more than once

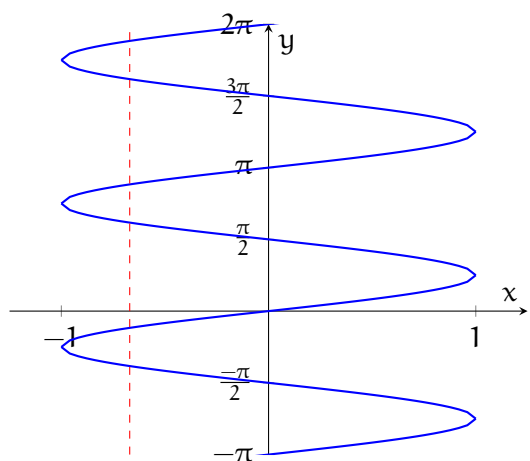


Figure 5.2: The inverse of an unrestricted sin function fails the vertical line test

5.1 Derivatives of Inverse Trigonometric Functions

f	f'
$\arcsin x$	$\frac{1}{\sqrt{1-x^2}}$
$\arccos x$	$-\frac{1}{\sqrt{1-x^2}}$
$\arctan x$	$\frac{1}{1+x^2}$
$\operatorname{arccsc} x$	$-\frac{1}{x\sqrt{x^2-1}}$
$\operatorname{arcsec} x$	$\frac{1}{x\sqrt{x^2-1}}$
$\operatorname{arccot} x$	$-\frac{1}{1+x^2}$

5.2 Practice

Exercise 1

Find the f' . Give your answer in a simplified form.

- $f(x) = \arctan x^2$
- $f(x) = x \operatorname{arcsec}(x^3)$
- $f(x) = \arcsin \frac{1}{x}$

Working Space

Answer on Page 43

Introduction to Graphs

6.1 Introduction

Some data is easier to work with if we imagine it as a set of connections. When we say *graph* in this chapter, we are referring to set of data defined by a collection of *nodes* (you may also hear vertex) and connected by *edges*. For example, on some social networks, each user can follow any number of other users. We can think of each user as a node, and the edge points from the user who follows to the user they follow:

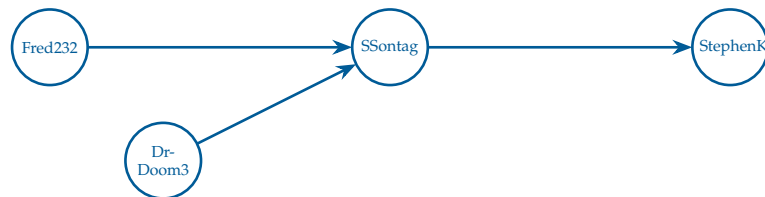


Figure 6.1: A graph showing four users and three “following” relationships.

Following is a *directed* relationship: Fred232 follows SSontag, but SSontag doesn’t follow Fred232. So we would say that this is a *directed graph* with four nodes and three edges.

There are also undirected graphs. For example, you can imagine a graph that represents big data lines between cities. All the big data lines allow communications in both directions, like two-way roads:

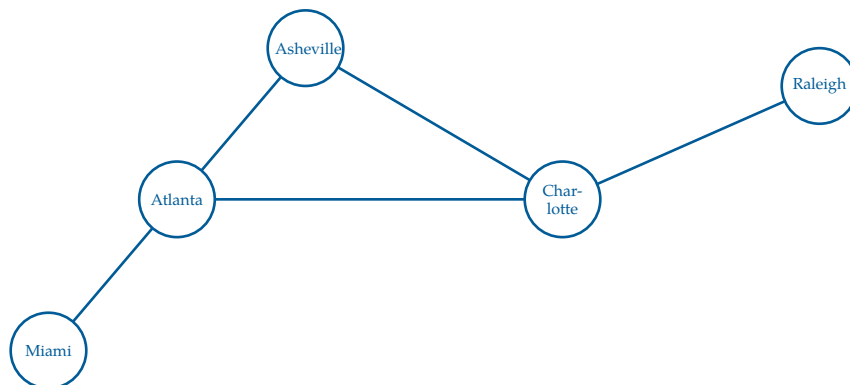


Figure 6.2: A set of cities, represented by nodes, on the east coast of the United States connected by roads, represented by edges.

The arrows are gone; if data can flow from Miami to Raleigh, then data can flow from Raleigh to Miami. A linked connection of edges between two or more nodes is called a *path*. Notice how Atlanta and Charlotte have 3 edges; we would say that Atlanta and Charlotte each have a *degree* of 3.

There is a whole branch of mathematics called *Graph Theory* that studies the properties of graphs. Here are two questions that mathematicians might ask about this graph:

- What is the smallest number of edges that we would need to follow to get from Miami to Raleigh?
- Does the graph have any paths where you could end up where you started?

That second question asks if there is a *cycle* in the graph. This graph has one cycle: Atlanta $\rightarrow$ Asheville $\rightarrow$ Charlotte $\rightarrow$ Atlanta.

Some graphs are *connected*: You can get from one node to any other node by following edges. Is this graph connected?

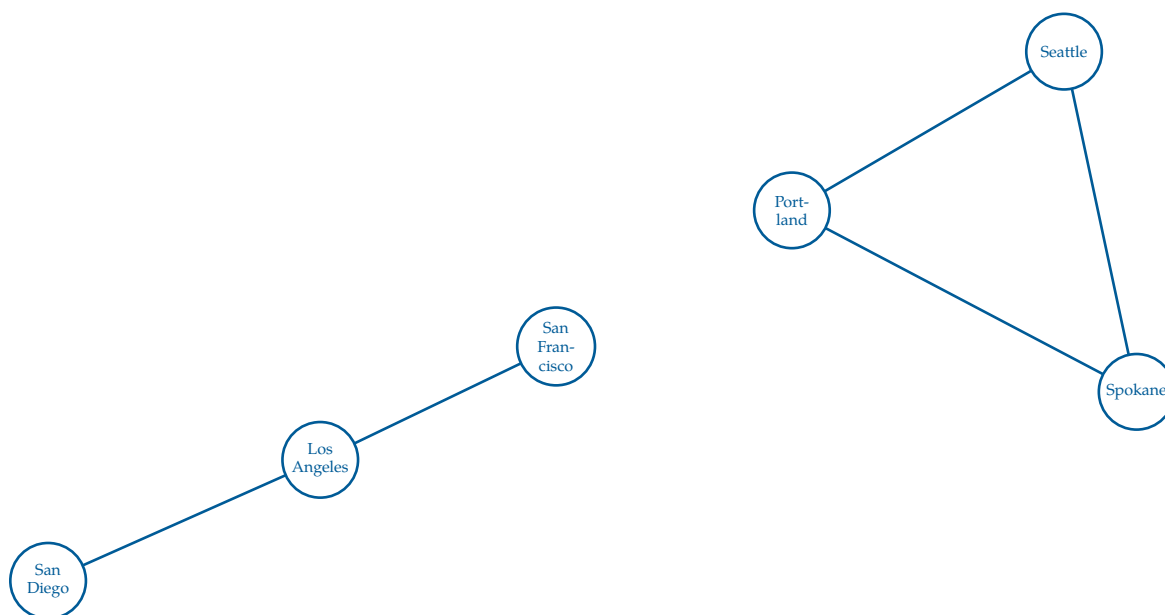


Figure 6.3: A set of cities (nodes) that are not all connected. Each set of *connected* cities forms a *connected component*.

This graph is *not* connected! You can't follow edges from San Diego to Seattle. However, you could identify two *connected components*, *subgraphs* of a graph that remain connected.

- San Diego, Los Angeles, San Francisco
- Portland, Seattle, Spokane

Connectivity is defined as, for all pairs of nodes in that graph, there exists a path between them. A subgraph is much simpler than a connected component, it's only requirement is to contain a subset of at least one node and, optionally, edges to go between nodes. Note that you cannot have a subgraph that contains a single node and an edge, because the edge would not be able to connect to anything else.

In graph data, the nodes and edges often have attributes. For example, a node representing a city might have a name and a population. An edge representing a data line might have a bandwidth (bits per second) and a latency (how many nanoseconds between when you put a bit into the pipe and when it comes out the other end). This is how we define a graph.

Exercise 2 Word Graphs

Working Space

Consider the concept of a graph as a set of nodes and edges. How would you create a graph where the nodes are words and the edges represent a one letter difference between two words?

For example, the word cat would be connected to cot, cart, and cut, but not to dog.

Define explicitly what the nodes would be in this graph, and what the edges would be. Draw a simple example of this graph with at least 5 nodes and 5 edges.

Answer on Page 43

6.2 Finding Good Paths

For many problems, we are trying to find the best path from one node to another. If all the edges are the same, this usually means finding the path that requires walking the fewest edges.

Sometimes the edges have a cost attribute. For example, you might want to find the

cheapest way to ship a container from New York City to Long Beach, California. In this case the nodes are train depots. Each train line between the depots has a cost. What is the cheapest path?

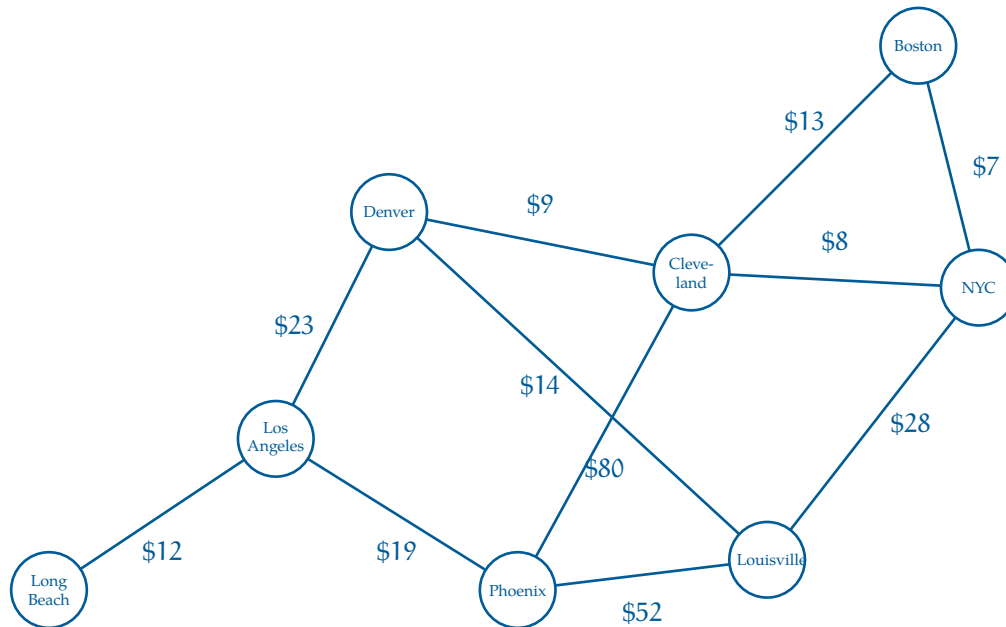


Figure 6.5: A graph of interconnected cities where each edge has a respective cost.

When edges have costs like this, we call the *weighted edges*, which forms a *weighted graph*. In weighted graphs, the *degree* of a node is split into the *in-degree* and *out-degree* of nodes. The in-degree identifies the incoming edges that originate from other nodes, and out-degree is the amount of edges going out of that node.

The graphs that you see here are really small, so finding efficient paths isn't difficult — you could just try all of them! However, in many computer programs, we are working with *millions* of nodes and edges. Efficient graph algorithms are *really* important.

6.3 Graphs in Python

There are many ways to represent graph data. There are database systems that are specifically designed to hold and analyze graph data. Not surprisingly, these are called *Graph Databases*.

You may also hear about *adjacency lists*, which store, for each node, the list of nodes it is connected to. Each direct connection from one node to another is called a *neighbor*. Simply, adjacency lists store lists of neighbors.

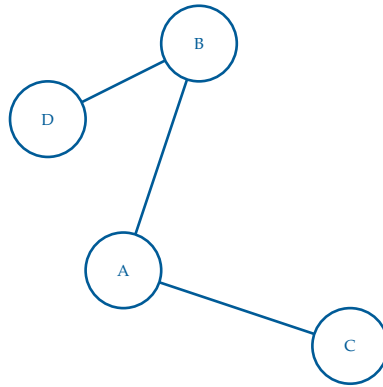


Figure 6.6: A simple (undirected unweighted) graph used to make an adjacency list.

As an adjacency list, this would look like:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

Exercise 3 Directed Adjacency Lists

Working Space

Consider the concept of adjacency lists discussed above. How would the storage method change if the edges were to have weights?

Answer on Page 44

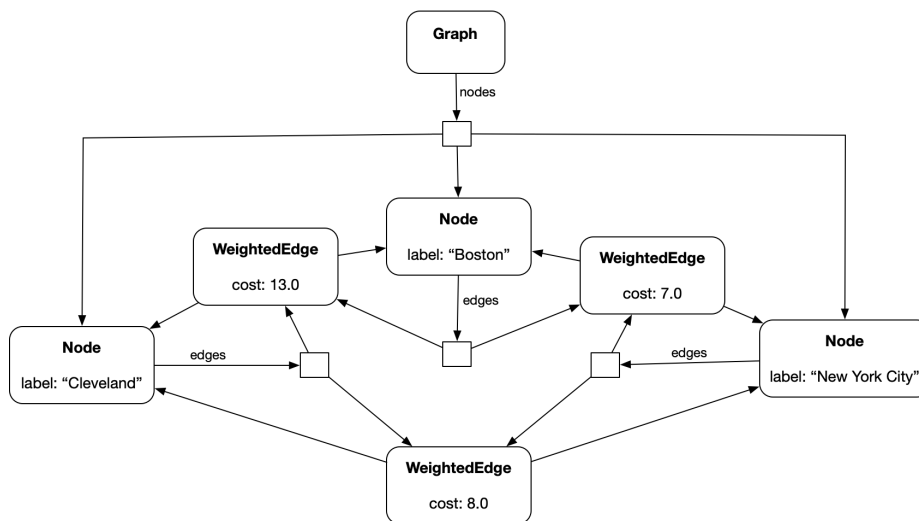
For simplicity, you are going to write Python classes that will let you represent an undirected graph with weighted edges, like the shipping problem above.

(Naturally, things would look a little different if the graph were directed or the edges were unweighted, but this is a good starting place.)

Create a file called `graph.py`. This will hold the code for your `Node` and `WeightedEdge` classes. We will also create a `Graph` class that will just hold onto the list of its nodes.

- A `Node` will have a label string and a list of edges that touch it.
- A `WeightedEdge` will have a cost and two nodes: `node_a` and `node_b`.
- A `Graph` will have a list of nodes.

Here is what the object diagram would look like if you had only three cities:



Put this code into `graph.py`

```

1 class Node:
2     def __init__(self, label):
3         self.label = label
4         self.edges = []
5
6     def __repr__(self):
7         return f"(node:{self.label}, edges:{len(self.edges)})"
8
9 class WeightedEdge:
10     def __init__(self, cost, node_a, node_b):
11         self.cost = cost
12         self.node_a = node_a
13         node_a.edges.append(self)
14         self.node_b = node_b
15         node_b.edges.append(self)
16

```

```

17 def other_end(self, node_from):
18     if self.node_a == node_from:
19         return self.node_b
20     if self.node_b == node_from:
21         return self.node_a
22     raise ValueError("node is not an endpoint of this edge")
23
24 class Graph:
25     def __init__(self):
26         self.nodes = []
27
28     def add_node(self, new_node):
29         self.nodes.append(new_node)
30
31     def __repr__(self):
32         return f"(Graph:{self.nodes})"

```

This is a graph class that handles undirected weighted graphs. Now, let's create some instances of Node and WeightedEdge and wire them together. Create another file in the same directory called cities.py. Put in this code:

```

1  import graph
2
3  # Create an empty graph
4  network = graph.Graph()
5
6  # Create city nodes and add to graph
7  long_beach = graph.Node("Long Beach")
8  network.add_node(long_beach)
9  los_angeles = graph.Node("Los Angeles")
10 network.add_node(los_angeles)
11 denver = graph.Node("Denver")
12 network.add_node(denver)
13 phoenix = graph.Node("Phoenix")
14 network.add_node(phoenix)
15 louisville = graph.Node("Louisville")
16 network.add_node(louisville)
17 cleveland = graph.Node("Cleveland")
18 network.add_node(cleveland)
19 boston = graph.Node("Boston")
20 network.add_node(boston)
21 nyc = graph.Node("New York City")
22 network.add_node(nyc)
23
24 # Create edges
25 graph.WeightedEdge(12, long_beach, los_angeles)
26 graph.WeightedEdge(23.0, los_angeles, denver)
27 graph.WeightedEdge(19, los_angeles, phoenix)
28 graph.WeightedEdge(52, phoenix, louisville)
29 graph.WeightedEdge(14, denver, louisville)

```

```
30 graph.WeightedEdge(80, phoenix, cleveland)
31 graph.WeightedEdge(9, denver, cleveland)
32 graph.WeightedEdge(8, cleveland, nyc)
33 graph.WeightedEdge(28, louisville, nyc)
34 graph.WeightedEdge(7, nyc, boston)
35 graph.WeightedEdge(13, cleveland, boston)
36
37 print(network)
```

Run it:

```
1 python3 cities.py
```

You should see some rather unexciting output:

```
1 (Graph:[(node:Long Beach, edges:1), (node:Los Angeles, edges:3), (node:Denver,
   ↪ edges:3),
2 (node:Phoenix, edges:3), (node:Louisville, edges:3), (node:Cleveland, edges:4),
3 (node:Boston, edges:2), (node:New York City, edges:3)])
```

But do not worry; we will make it more exciting in the next chapter!

Laws, Paradoxes, Thought Experiments, and the Illusions

A *paradox* is a statement that contradicts itself or defies intuition. Paradoxes often arise in mathematics, physics, and philosophy, challenging our realities and understanding of reality and logic.

We are going to talk about them here to introduce the idea of proofs by contradiction, which is a powerful tool in mathematics. A proof by contradiction assumes that the statement we want to prove is false and then shows that this assumption leads to a logical contradiction. We are also going to talk about a multitude of paradoxes in fields such as mathematics, physics, computer science, and architecture.

Some of these will also be *thought experiments*. Thought experiments involve considering an imaginary scenario that tests a theory or argument, and many times, one that would be rare or impossible to test. Scientists use this to refute past (incorrect) knowledge, establish new theories, or even provide scenarios about human nature¹. Doing this allows engineers to isolate variables, look at edge cases, improve intuition of what *should* happen, and consider how breakthroughs could occur. In fact, Isaac Newton used a thought experiment comparing a falling apple and the Moon to help develop his theory of universal gravitation (recall the Newton's cannon idea). In many of our physics scenarios, we imagine a *frictionless surface*. This will never truly exist, all materials involve some sort of friction, so we consider this a thought experiment.

Further, some statements which this chapter explores are laws of science and mathematics. Some of these laws are not intuitive,

If you recall, we talked in the proofs chapter about the different types of proofs, specifically proofs by contradiction. We will now talk about some examples of proofs by contradiction, and how they relate to paradoxes. We will use proofs by contradiction to prove many of our paradoxes.

The Barber's Paradox is a self-referential paradox that arises in set theory and logic.

The paradox is often stated as follows

Theorem 1. *In a town, there is a barber who shaves all and only those men who do not shave themselves. The question is: Does the barber shave himself?*

¹https://en.wikipedia.org/wiki/Trolley_problem

Well there are two cases to consider:

- If the barber shaves himself, then according to the definition, he is now someone who *shaves himself*. The rule says the barber only shaves people who do not shave themselves. This is a contradiction because the barber cannot both shave himself and not shave himself at the same time.
- If the barber does not shave himself, then according to the definition, he is someone who does not shave himself. The rule says the barber shaves all those who do not shave themselves. This means the barber must shave himself, which again leads to a contradiction because he cannot both shave himself and not shave himself at the same time.

The reason this is a paradox is that no matter which path you take, you end up at a contradiction. The barber cannot exist under those rules. A paradox will often arise from statements that are self-referential or involve circular definitions, which is the case here. It is important to note that some paradoxes that we will talk about will have resolutions, but the Barber's Paradox is an example of a paradox that does not have a resolution.

In proofs by contraction we often intentionally create a contradiction to prove a theorem. In the Barber's Paradox, we assumed the two possible cases for the barber (shaving himself or not shaving himself) and showed that both lead to a contradiction.

7.1 Mathematical Paradoxes

7.1.1 Russell's Paradox

Russell's Paradox is a paradox within set theory. It arises from the statement that

Theorem 2. *Consider the set of all sets that do not contain themselves. Does this set contain itself?*

Let's consider some simple sets. The set of all apples *is not* an apple, it is a list of apples. Similarly, the set of all of ideas *is* an idea, it is a concept. For Russell's statement, let's create a master set R that contains only the sets that do not contain themselves. There are two cases:

- If R is not a member of itself, then its definition entails that it is a member of itself.
 $\rightarrow \leftarrow$
- If R is a member of itself, then it is not a member of itself, since it is the set of all sets that are not members of themselves. $\rightarrow \leftarrow$

Using symbols from our sets chapter,

Let $R = \{x | x \notin x\}$. Then $R \in R \iff R \notin R$. This is a $\rightarrow \leftarrow$

7.1.2 Simpson's Paradox

Simpson's Paradox is a phenomenon in statistics where a trend or correlation that appears in several different groups of data can disappear or reverse when the groups are combined. Let's look at an example.

Imagine we are comparing two hospitals, Hospital A and Hospital B, for their success rates in completing an operational surgery. We have the following data:

Hospital	Total Patients	Survived	Survival Rate
Hospital A	1000	800	80%
Hospital B	1000	890	89%

From this, we see that the same number of patients were treated at both hospitals, but Hospital B has a higher survival rate. This would lead us to think that Hospital B is the better hospital, due to money, education of doctors, or whatever factor. However, let's look at the data when separated by those with *high-risk* surgeries versus those with *low-risk* surgeries.

Patients that had low-risk surgeries, or were in good health, are given by following data

Hospital	Patients	Survived	Survival Rate
Hospital A	100	98	98%
Hospital B	900	850	94.4%

And those with poor health and high-risk surgeries:

Hospital	Patients	Survived	Survival Rate
Hospital A	900	702	78%
Hospital B	100	40	40%

Looking at the data based on the health categorization, Hospital A is actually better in both categories. This "paradox" occurs due the unaccounted, **independent variable**: the incoming health of the patient. Hospital A took on the harder cases (90% of their patients are high-risk). Hospital B, although it initially was thought to be the better hospital, mainly took on those with good health (90% were low-risk).

For fun, let's plot this using pandas and matplotlib libraries in Python. The code is as follows:

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # aggregate the data
6 data = {
7     'Condition': ['Good', 'Good', 'Poor', 'Poor', 'Total', 'Total'],
8     'Hospital': ['A', 'B', 'A', 'B', 'A', 'B'],
9     'Survival': [0.98, 0.94, 0.78, 0.40, 0.80, 0.89]
10 }
11 df = pd.DataFrame(data)
12
13 # split into subgroups and totals for showing different correlations
14 subgroups = df[df['Condition'] != 'Total']
15 totals = df[df['Condition'] == 'Total']
16
17
18 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5), sharey=True)
19 # regression lines and plots
20 ax1.plot([-0.2, 0.8], [0.98, 0.78], 'b--', marker='o', label='Trend A')
21 ax1.plot([0.2, 1.2], [0.94, 0.40], 'y--', marker='o', label='Trend B')
22 ax1.legend()
23 ax2.plot([-0.2, 0.2], [0.80, 0.89], 'k-', marker='o', label='Overall Trend')
24 ax2.legend()
25 sns.barplot(data=subgroups, x='Condition', y='Survival', hue='Hospital', ax=ax1)
26 ax1.set_title('Subgroups (A Wins)')
27
28 sns.barplot(data=totals, x='Condition', y='Survival', hue='Hospital', ax=ax2)
29 ax2.set_title('Total (B Wins)')
30
31 plt.tight_layout()
32 plt.show()

```

This creates two bar plots with opposing regression lines:

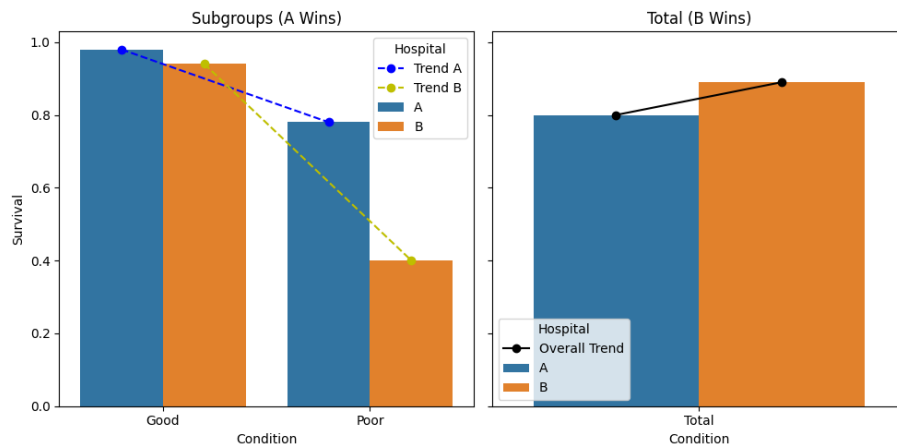


Figure 7.1: Simpson's Paradox: Subgroups vs. Total

7.1.3 Monty Hall Problem

Watch the following video the introduce the [Monty Hall Problem](#).

The problem is as follows:

Theorem 3. *You are on a game show and are given the choice of three doors. Behind one door is a car; behind the others, goats. You pick a door, say No. 1, and the host, who knows what's behind the doors, opens another door, say No. 3, which has a goat. He then says to you, "Do you want to switch to door No. 2?" Is it to your advantage to switch your choice?*

ADD: an adobe illustration of the doors and the host opening one of them, and the probabilities of each door having the car after the host opens one.

Your choice of Door No. 1 has a $1/3$ chance of being the car and a $2/3$ chance of being a goat. When the host opens Door No. 3, which has a goat, the probability of each door changes! Even though there are two doors left, the probability is $50/50$. The probability of the car being behind Door No. 1 remains $1/3$, while the probability of the car being behind Door No. 2 *increases to* $2/3$. Therefore, it is to your advantage to switch to Door No. 2, as it has a higher probability of containing the car.

7.1.4 Birthday Problem

The birthday problem is a famous problem in probability and combinatorics. It goes as follows:

Theorem 4. *In a group of n people, what is the probability that at least two people share the same birthday?*

The answer is often surprising to people because it is much higher than most people expect. Only 23 people are needed to guarantee a *majority*, where greater than 50% chance that at least two people share a birthday. Let's prove this.

Let $P(A)$ the probability that a group of k people do *not* share a birthday. Further, let $P(B) = \neg P(A) = 1 - P(A)$ be the probability that at least two people share a birthday. We want to find the smallest k such that $P(B) > 0.5$, as this constitutes a majority.

If there are only two people in a room (meaning $k = 2$), the probability that there does not exist a shared birthday is $P(A) = \frac{364}{365}$, meaning the probability that there is a shared birthday is $P(B) = 1 - P(A) = \frac{1}{365}$. If there are three people in a room, the probability that there does not exist a shared birthday is $P(A) = \frac{364}{365} \cdot \frac{363}{365}$, meaning the probability that there is a shared birthday is $P(B) = 1 - P(A) = 1 - \frac{364}{365} \cdot \frac{363}{365}$.

This pattern continues, and we can generalize the probability that there does not exist a shared birthday for k people as follows:

$$P(A) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365 - k + 1}{365} = \prod_{i=0}^{k-1} \frac{365 - i}{365}.$$

Therefore, the probability that there is at least one shared birthday is:

$$P(B) = 1 - P(A) = 1 - \prod_{i=0}^{k-1} \frac{365 - i}{365}.$$

We now seek the smallest k such that

$$1 - \prod_{i=0}^{k-1} \frac{365 - i}{365} > 0.5.$$

Equivalently,

$$\prod_{i=0}^{k-1} \frac{365 - i}{365} < 0.5.$$

Computing this product gives

$$P(A) \approx 0.5073 \quad \text{when } k = 22,$$

so

$$P(B) \approx 1 - 0.5073 = 0.4927 < 0.5.$$

But for $k = 23$,

$$P(A) \approx 0.4927,$$

so

$$P(B) \approx 1 - 0.4927 = 0.5073 > 0.5.$$

Thus, the smallest number of people needed so that the probability of at least two sharing a birthday exceeds 50% is 23 people. Much smaller than most people expect!

7.1.5 Zeno's Paradoxes

Zeno was a Greek philosopher who is famous for his paradoxes, who created challenging thought experiments and paradoxes that question our understand of motion, space, physics, and time. Zeno's paradoxes are often used to illustrate the concept of *infinity* and the nature of space and time.

The Dichotomy Paradox

His most famous paradoxes include is called the *Dichotomy Paradox*, which he stated as

That which is in locomotion must arrive at the half-way stage before it arrives at the goal.

This paradox suggests that before an object can reach its destination, it must first reach the halfway point, and before it can reach the halfway point, it must reach the quarter point, and so on, leading to an infinite number of steps that must be completed before reaching the destination. This seems to imply that motion is impossible, as it would require completing an infinite number of tasks in a finite amount of time.

So then, how is motion possible?

The resolution to Zeno's paradoxes lies in the concept of *limits* and *infinite series*. In modern mathematics, we understand that an infinite number of steps can be completed in a finite amount of time if the steps get smaller and smaller. If we consider the remaining distance to be covered as 1 unit, the first step is $\frac{1}{2}$, the second step is $\frac{1}{4}$, the third step is $\frac{1}{8}$, and so on. The total distance covered after n steps can be expressed as a *geometric series*:

$$S = \sum_{n=1}^{\infty} \frac{1}{2^n} = 1$$

For proof's sake, let

$$S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots$$

Then, multiplying both sides by $\frac{1}{2}$ gives

$$\frac{1}{2}S = \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \cdots$$

$$S - \frac{1}{2}S = \frac{1}{2}$$

$$\frac{1}{2}S = \frac{1}{2} \implies S = 1$$

7.2 Physics and Time Travel Paradoxes

A lot of what we have touched are counter-intuitive mathematical equations. Now we can look at some physics paradoxes and thought experiments that may mess with your brain a bit. A lot of these actually involve a concept called superposition.

7.2.1 Schrödinger's Cat

Erwin Schrödinger was a physicist with thoughts on quantum superposition, subatomic events, and early quantum theory. This thought experiment arose in a discussion with Albert Einstein, Niels Bohr, and Werner Heisenberg, all physicists from Europe working towards quantum development.

The thought experiment is as follows:

Schrödinger imagined placing a cat in a sealed box with a device triggered by a quantum event (such as the decay of an atom, which depends on an electron-level process, or radioactive gunpowder, which Einstein preferred). The event has a 50% chance of happening and a 50% chance of staying dormant. If the event occurs, the cat dies; if not, it lives. However, you do not know whether the cat is alive or not until the box is opened. Schrödinger states that, the moment before the box is opened, the cat is equal parts dead and alive.

Why is this? Well, the cat, in Schrödinger's world, is theoretically governed by quantum physics and quantum position. **Quantum Superposition** states that an object, usually an electron, exists in a **superposition of states**, such that it occupies multiple positions or energy states until it is measured. Before measurement, the electron is not in one definite state; instead, it is described by a wavefunction that encodes multiple possibilities simultaneously. If electrons are fired at a barrier with two narrow slits and then detected on a screen behind it, they produce an interference pattern, one with alternating covered and uncovered regions. This is characteristic of wave behavior, showing that electrons are travelling in waves. Further, when we talk about nonpolar covalent bonds, the electrons are present in both atoms simultaneously.

We say that the cat is in a superposition of both equally possible dead or alive states. There is a great video on Schrödinger's Cat and its relation to quantum physics by TedEd in your digital resources, check it out!

Arrow Paradox

A very similar problem on superposition of an arrow comes from Zeno (who we talked about earlier) is the *Arrow Paradox*:

If everything when it occupies an equal space is at rest at that instant of time, and if that which is in locomotion is always occupying such a space at any moment, the flying arrow is therefore motionless at that instant of time and at the next instant of time but if both instants of time are taken as the same instant or continuous instant of time then it is in motion.

In short, this is a fancy way of saying that if we look at an arrow in flight at any single instantaneous moment, it will appear to be at rest, and it occupies a specific position in space. In this tiny instant of time (recall dt from calculus), the arrow is neither moving to where it is, nor to where it is not. This seems to suggest that the arrow is motionless at every instant of time, and therefore, it cannot be in motion.

However, the solution to this paradox lies in the concept of limits. Motion is not just about being at a specific position at an instant, but about the change in position over time. The arrow's motion can be understood as a continuous process, where its position changes smoothly over time, even though at any single instant it may appear to be at rest, just as single points on a curve can be thought of as having no length, but the curve itself can have a infinite length.

7.2.2 Time Travel Paradoxes

While we know (as of writing this chapter) that time travel is not real, it is fun to think about and arises in pop culture such as movies a lot! Time travel paradoxes are thought experiments which arise from the contradiction that arises within time travel or the foreknowledge of the future. Time travel raises logical problems because it allows events to influence their own causes.

The most well-known example is the **grandfather paradox**. Suppose a person travels back in time and prevents their grandfather from having children killing their grandfather. This would mean the time traveler is never born, as one of the time traveler's parents are never conceived. The time traveler, then, could not have gone back in time to interfere in the first place. The sequence of events becomes logically inconsistent: the same action both must and must not occur. This is a contradiction ($\rightarrow\leftarrow$), as we talked about, and many

people use this as an argument to prove that time travel cannot logically exist.

Another time travel paradox is preventing Abraham Lincoln's assassination. Imagine a time traveler goes back to 1865 and prevents Lincoln's assassination. At first glance, this seems like a straightforward intervention. However, if Lincoln survives, the historical conditions that motivated the time traveler to go back, such as learning about the assassination in the first place, no longer exist. This creates a contradiction: if Lincoln was never assassinated, why would the traveler have made the journey to stop it?

More generally, these are consistency paradoxes, where altering the past creates contradictions in the timeline. In physics, these scenarios contradict our existing knowledge, meaning either constraints exist to preserve consistency, or our understanding of time and causality is incomplete.

7.3 Architecture and Geometry Paradoxes

Architecture and geometric paradoxes come with their own set of paradoxes, often warping visual perception and challenging our understanding of space and geometry.

7.3.1 Penrose Steps

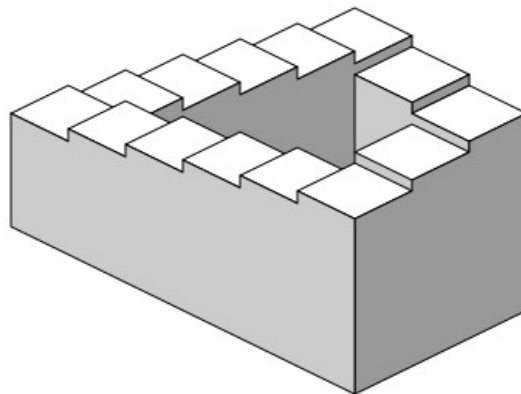


Figure 7.2: Penrose steps diagram from Wikipedia - By Sakurambo - Own work, Public Domain, <https://commons.wikimedia.org/w/index.php?curid=794844>

The penrose steps are an architectural paradox that creates the illusion of an infinite staircase.² The steps appear to be continuously ascending or descending, but in reality, they

²Christopher Nolan famously reconstructed this illusion in his film *Inception* (2010). If you are interested, the scene can be found [here](#) and behind the scenes [here](#).

form a closed loop. This paradox is achieved through clever use of perspective and geometry, creating an optical illusion that tricks the viewer's perception of space.

7.3.2 Mobius Strip

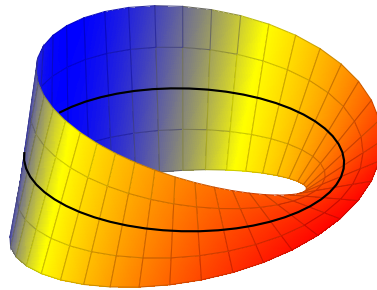


Figure 7.3: A parametric graph of a mobius strip.

The mobius strip is a piece of paper is a *surface* that is said to have one side and one boundary. It is formed most easily (you can do this at home!) by taking a strip of paper, giving it a half twist, and then taping the ends together. The resulting shape is a non-orientable surface, meaning that it has only one side and one edge. If you were to start drawing a line down the middle of the strip, you would find that you return to your starting point after traversing the entire length of the strip, without ever lifting your pen. There is no *inside* or *outside* to the mobius strip.

If you are curious, the mobius strip is defined by the following parametric equations:

$$\mathbf{r}(u, v) = \left((1 + v \cos \frac{u}{2}) \cos u, (1 + v \cos \frac{u}{2}) \sin u, v \sin \frac{u}{2} \right)$$

where u ranges from 0 to 2π and v ranges from -1 to 1 . Do not worry, this is not something you need to know how to do at all! We will cover parametric in a future workbook.

7.3.3 Coastline paradox

The *coastline paradox* highlights an issue in measuring irregular geometric objects; their length is changes under changes in measurement scale. Unlike smooth curves in classical Euclidean geometry, natural coastlines exhibit intricate, self-similar structure across many scales. As the measuring unit becomes smaller, more of this fine structure is resolved, and the total measured length increases. This originated in the 1950's when a Lewis Fry Richardson was researching how border lengths could effect the likelihood of war, and there was a discrepancy between the Portuguese measurement of Spain's border compared to the Spanish. Simply, the shorter the unit (or ruler), the longer the measured border; the Spanish and Portuguese geographers were simply using different rulers. However, as

the “ruler length” approaches zero, the length grows infinity, rather than approaching a finite value!

ADD: Diagram from tikz of coastline

This phenomenon contradicts with calculus and its ideas on the definition of arc length. In calculus, the length of a curve ($y = f(x)$) over an interval is defined as the definite integral $\int f(x) dx$ between two points, which often leads to a finite value for smooth or simple functions. The coastline paradox demonstrates that this limiting process fails for highly irregular sets; the coastlines are not “smooth” enough. Consequently, arc length integrals cannot be utilized, and alternative methodologies are used to find the length of these areas, termed fractal surface perimeters.

7.4 Computer Science Paradoxes and Laws

7.4.1 Moravec’s Paradox

Hans Moravec, a pioneer in robotics and Artificial Intelligence, wrote in 1988,

it is comparatively easy to make computers exhibit adult level performance on intelligence tests or playing checkers, and difficult or [near] impossible to give them the skills of a one-year-old when it comes to perception and mobility

What he means by this statement, referred to as the *Moravec’s Paradox* is that tasks that are *easy* for humans are *hard* for machines, and tasks that are *hard* for humans are relatively *easy* for machines.

A good example of this is chess. Chess robots have been being developed since the 1980’s, and in 1997, a machine called Deep Blue by IBM defeated a world champion grandmaster in chess. Typically, chess has been a level of skill attributed to the most intelligent humans. However, contrary to what would arbitrarily think, chess is one of the easiest tasks for a computer to do.

Along with chess comes proving mathematical theorems, brute-force algorithms, multiplying large numbers, or even integration, all of which were generally thought of as difficult tasks to humans but for computers are the most simple. Why is this the case? Well, simply put, humans have skills that have naturally become second nature, or unconscious, to us, like movement, recognizing a friend’s face, settings goals, etc. Humans naturally want to automate that which is not in tune with this unconscious behavior, like solving integrals, identifying the best path to hit all capitals of the United States, or even just take the best billiards shot.

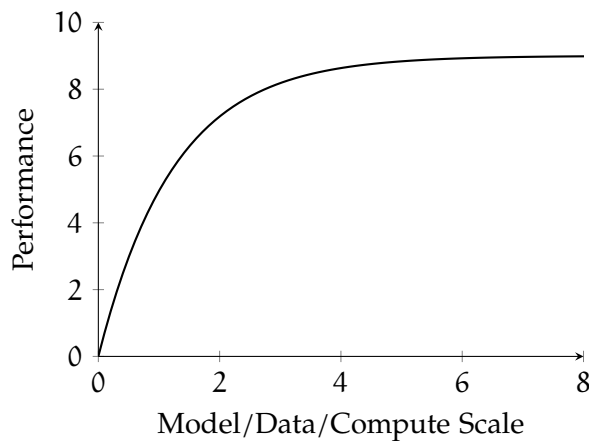
Intuitively, this means the best way for humans to learn is to practice cognitive skills like

reading carefully, practicing problems, or explaining ideas, in order to continue gaining mastery through repetition. Recognizing patterns or spotting errors takes practice and is harder to master for humans, but it takes human-centric skills to maintain and develop.

7.4.2 Neural Scaling Law

Imagine you are working for a factory that is producing pencils. With a single, static, automatic assembly line, you make pencils at a rate of 100 pencils per hour. By adding a second assembly line, you are now able to make pencils at 200 pencils per hour, double the rate it was previously. This shows that generally, production scales *linearly*.

However, not all systems scale this cleanly. Suppose instead that each additional assembly line becomes harder to integrate: workers need to coordinate more, machines interfere slightly, and space becomes constrained. Now, when you add a second line, production might increase to 180 pencils per hour instead of 200. Adding a third might bring you to 250, not 300. The total output still increases, but each new addition contributes less than the previous one. This is called *diminishing returns*.



Neural scaling laws behave more like this second scenario. Say you have a neural network model with a set of training params N , D , C , L , which stand for number of parameters, size of dataset, cost of computing, and a loss function, respectively. As you increase a model's size, the amount of data it sees, or the compute used to train it, performance improves steadily, not linearly. Early increases lead to large gains, while later increases produce smaller, more incremental improvements. Importantly, the improvement is still predictable and consistent, which is why researchers can estimate how much better a system will perform if they scale it up. Consequently, just overloading the model with a higher, almost infinite, magnitude of parameters will lead to a plateau of performance. The graph of performance vs parameters looks similar to an upside-down hockey stick, it scales really well for a while when parameters are being increased up to a point, but then it gradually decreases in rate of improvement of time.

Answers to Exercises

Answer to Exercise 1 (on page 20)

1. By the chain rule, $f'(x) = 2 \arctan x \times \frac{d}{dx} \arctan x = 2 \arctan x \frac{1}{1+x^2}$
2. By the Product rule, $f'(x) = x \frac{d}{dx} \operatorname{arcsec}(x^3) + \operatorname{arcsec}(x^3)$. Further, by the chain rule, $\frac{d}{dx} \operatorname{arcsec}(x^3) = \frac{1}{(x^3)\sqrt{(x^3)^2-1}} \times \frac{d}{dx}(x^3) = \frac{3x^2}{x^3\sqrt{x^6-1}}$. Therefore, $f'(x) = \frac{3}{\sqrt{x^6-1}} + \operatorname{arcsec}(x^3)$
3. By the chain rule, $f'(x) = \frac{1/x}{\sqrt{1-(1/x)^2}} \times -\frac{1}{x^2} = -\frac{1}{x^3\sqrt{1-\frac{1}{x^2}}}$

Answer to Exercise 2 (on page 23)

The nodes in this graph are unique words. The edges represent a valid one letter operations that can transform one word into another word. These operations include adding a letter, removing a letter, or changing a letter into another. For example, the word cat would be connected to cot (change), cart (add), and cut (change), but not to dog. Usually this will be represented as a tree, which is a special type of graph we will cover later.

Here is an example of a word graph:

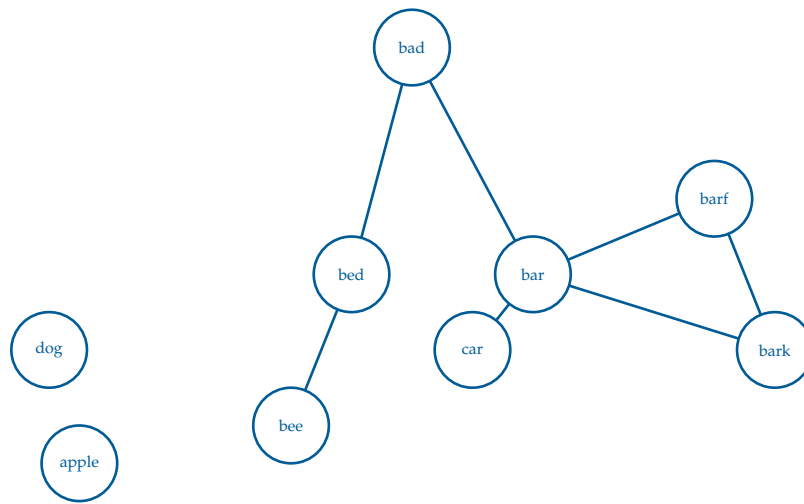


Figure 6.4: A word graph where the nodes are unique words and the edges represent a one letter operations between two words.

Answer to Exercise 3 (on page 25)

In a weighted graph, each node would have to store a set of neighbors and their respective weights. The best way to do this would be a tuple. For example,

- A: [(B, 4), (C, 3)]
- B: [(A, 4), (D, 8)]
- C: [(A, 3)]
- D: [(B, 8)]



INDEX

- adjacency lists, [24](#)
- arrow paradox, [37](#)
- coastline paradox, [39](#)
- compound interest, [7](#)
- compound interest formula, [8](#)
- connected, [22](#)
- connected component, [22](#)
- connected components, [22](#)
- cycle, [22](#)
- degree, [22](#)
- diminishing returns, [41](#)
- directed, [21](#)
- directed graph, [21](#)
- e, [13](#)
- edges, [21](#)
- exponential decay, [17](#)
- exponential growth, [8](#)
- exponents, [3](#)
 - fractions, [4](#)
 - negative, [4](#)
 - zero, [4](#)
- geometric series, [35](#)
- grandfather paradox, [37](#)
- graph, [21](#)
 - connected, [22](#)
 - directed, [21](#)
 - undirected, [21](#)
- Graph Databases, [24](#)
- Graph Theory, [22](#)
- half-life, [16](#)
- Hans Moravec, [40](#)
- in-degree, [24](#)
- infinite series, [35](#)
- limits, [35](#)
- ln, [13](#)
- log, [11](#)
 - in python, [11](#)
- logarithm, [11](#)
 - change of base, [13](#)
 - identities, [12](#)
 - natural, [13](#)
- mobius strip, [39](#)
- Monty Hall Problem, [33](#)
- Moravec's Paradox, [40](#)
- neighbor, [24](#)
- nodes, [21](#)
- out-degree, [24](#)
- paradox, [29](#)
- path, [22](#)
- penrose steps, [38](#)
- radioactive decay, [16](#)
- Russell's Paradox, [30](#)

Schrödinger's cat, [36](#)

set theory, [30](#)

Simpson's paradox, [31](#)

subgraphs, [22](#)

superposition, [36](#)

thought experiments, [29](#)

time travel paradoxes, [37](#)

weighted edges, [24](#)

weighted graph, [24](#)